SEN381: CIVICCONNECT

Milestone 3 — Individual Contribution

Architecture Conformance, Quality Design, Risk Management and Release Readiness

Student: Bianca Grobler
Project: CivicConnect
Milestone: M3
Role: Architecture Conformance, Quality Design, Risk and Release Readiness Lead
Date: 10 October 2026

1. Introduction

1.1 Purpose

This document records my individual contribution to Milestone 3 (M3) of the CivicConnect project. My responsibilities focus on evaluating the implementation against the architectural decisions established during M2, maintaining traceability between decisions and implementation consequences, identifying implementation-stage risks, documenting architecture and design technical debt, specifying tests for status-transition business logic, and assessing release readiness.

My contribution is primarily concerned with architecture, quality assurance, risk analysis, documentation and verification planning. It supports the development team by making implementation deviations visible, identifying potential failures before release, and defining the evidence required to demonstrate that CivicConnect is ready for delivery.

1.2 Scope of my contribution

My assigned responsibilities are:

1. Review the implementation against the M2 architectural decisions and Architecture Decision Records (ADRs).
2. Document architectural deviations, their causes, revised decisions where applicable, and their consequences.
3. Update the ADRs with implementation-stage and later-consequence information.
4. Extend the existing Risk Register beyond the risks identified during M2.
5. Maintain the Architecture and Design Technical Debt Register.
6. Specify business-logic test cases for status transitions.
7. Consolidate release evidence and determine the appropriate release-readiness classification.
8. Document residual risks and prepare an individual defence explaining the reasoning behind my decisions.

The document distinguishes between the **reviewed M2 baseline**, the **required M3 implementation**, and **evidence that still needs to be collected**. This distinction is important because an architectural requirement or test specification is not proof that the corresponding implementation has been completed successfully.

2. Architecture Conformance Review

2.1 Review approach

The architecture-conformance review compares the documented architectural intentions with the implementation represented in the reviewed M2 project baseline.

The review follows these steps:

1. Identify the original architectural decision.
2. Establish the implementation evidence available in the reviewed baseline.
3. Compare the observed implementation with the intended architecture.
4. Record any deviation or missing evidence.
5. Explain the engineering consequences.
6. Identify the corrective action and evidence required for closure.

A deviation is not automatically evidence that an architectural decision was incorrect. It may indicate incomplete implementation, an enforcement problem, an undocumented change, or a decision that needs to be reconsidered in light of implementation experience.

2.2 ADR-M2-01: Layered application architecture

Original decision

CivicConnect uses a single deployable web application with logical separation between application responsibilities. The reviewed project structure includes the following folders within `CivicConnect.Web`:

- `Application`
- `Security`
- `Domain`
- `Data`

The intention is to separate business rules, domain concepts, security responsibilities and data access while keeping deployment and project management relatively straightforward.

The intended dependency direction is from the presentation layer towards application-level functionality and then towards domain or data-access responsibilities as appropriate. Business rules should not be duplicated across controllers, and data access should not be scattered throughout the application.

Reasoning behind the decision

A single deployable application is appropriate for a project of CivicConnect's scale because it reduces deployment complexity and allows the team to develop and test the system without maintaining multiple independently deployed services.

Logical layering also supports maintainability. A change to data retrieval should not require unrelated presentation code to be rewritten, and business rules should be reusable rather than repeated in multiple endpoints.

The disadvantage is that folders alone do not enforce architectural boundaries. Developers may still introduce unwanted dependencies unless these are prevented by project references, interfaces, code review, automated checks or other suitable controls.

Conformance assessment

Preliminary classification: Partially conformant; further M3 verification required.

The reviewed baseline contains the intended logical folders. However, logical separation does not by itself demonstrate that the boundaries are consistently enforced.

The review must therefore examine:

- Whether controllers contain business logic that belongs in the application or domain layer.
- Whether data access is performed through the intended components.
- Whether security decisions are centralised and consistently applied.
- Whether dependencies follow the documented direction.
- Whether new M3 functionality follows the same structure.
- Whether automated checks or code-review evidence demonstrate compliance.

The available baseline is not sufficient to claim that every architectural boundary is enforced.

Consequences and corrective action

If the layering is not consistently maintained, the application may develop tight coupling between controllers, business rules and data access. This makes changes harder to test and increases the risk of duplicated or inconsistent behaviour.

The recommended corrective actions are:

1. Review the M3 implementation against the existing folder and dependency structure.
2. Move duplicated business rules into appropriate application or domain components.
3. Keep data-access responsibilities in the intended data layer.
4. Record any deliberate architectural change in the relevant ADR.
5. Add or strengthen architectural checks where justified by project scope.
6. Verify the corrected structure during code review.

Alternatives considered

Alternative A: Keep a single application with logical layers.

This retains a simple deployment model while improving separation of responsibilities. It is suitable when the application is small enough to be managed as one deployable unit.

Alternative B: Split the application into multiple independently deployed services.

This can improve independent deployment and scaling, but introduces additional infrastructure, communication, monitoring and deployment complexity.

Decision: Retain the single-application approach unless specific requirements demonstrate that separate services are necessary. Strengthen internal boundaries before introducing distributed architecture.

Follow-up evidence

- M3 repository review.
- Review of controller, application, domain and data dependencies.
- Code-review records for architectural corrections.
- Automated architectural checks, if implemented.
- Updated ADR if the original decision has been formally revised.

2.3 ADR-M2-02: Status-transition business logic

Original decision

The M2 architectural direction identified `StatusTransitionService` as the central location for status-transition rules. The intended design uses state-specific behaviour to control which transitions are valid.

The reviewed M2 baseline did not demonstrate the complete `StatusTransitionService` and write-endpoint implementation required to establish that this design was fully implemented. The M3 implementation must therefore be reviewed directly before conformance can be confirmed.

Reasoning behind the decision

Status changes are business operations rather than simple field assignments. A status change can affect the validity of a record, the actions available to users, the history of the record and the information returned by the application.

If each controller or client can update a status independently, inconsistent rules may develop. For example, one endpoint could permit a transition that another endpoint rejects.

Centralising transition logic provides one place to:

- Define permitted transitions.
- Reject invalid transitions.
- Enforce the required order of operations.
- Keep status changes consistent across endpoints.
- Apply authorisation and validation before updating the record.
- Record the outcome of a transition where audit history is required.

Required implementation behaviour

The M3 implementation should ensure that:

1. A status change is checked against the current state.
2. The requested target state is valid for the current state.
3. The caller has the required permission.
4. Relevant business rules are satisfied.
5. Invalid transitions are rejected without partially updating the record.
6. Successful transitions persist the new state.
7. Audit history is recorded where required by the specification.
8. Concurrent changes do not silently overwrite one another.
9. API responses communicate success or rejection clearly.

The exact transition matrix must be taken from the approved CivicConnect requirements and implemented rules. It should not be invented from the test names alone.

Conformance assessment

Classification: Implementation evidence required.

The reviewed baseline does not, by itself, establish that the complete transition service and write endpoint exist or that the transition rules are enforced end to end.

The M3 review must establish:

- Whether the transition service exists.
- Whether all status updates pass through it.
- Whether direct status assignments bypass the service.
- Whether invalid transitions are rejected.
- Whether persistence and audit history are handled consistently.
- Whether role-based restrictions are applied.
- Whether concurrent requests are handled safely.

Alternatives considered

Alternative A: Central transition service using state-specific behaviour.

This approach encapsulates behaviour associated with each state and keeps transition decisions in one central business component. It is useful when states have different rules or behaviour.

Alternative B: Central transition table or explicit transition map.

This approach represents allowed transitions as a defined set of current-state and target-state combinations. It can be simpler to maintain where transition rules are relatively straightforward.

Alternative C: Direct status updates in controllers.

This is the least desirable option because it can duplicate rules and allow different endpoints to behave inconsistently.

Decision: Retain centralised transition handling. The implementation should use the approved M2/M3 design unless a documented ADR revision explains why another approach is preferable.

Corrective action

- Verify that all status-changing operations use the central transition component.
- Search for direct assignments to the status property outside the intended component.

- Confirm that validation occurs before persistence.
- Confirm that failed transitions leave the record unchanged.
- Confirm audit history requirements and verify their implementation.
- Add automated tests for valid transitions, invalid transitions, authorisation and persistence.
- Update the ADR with the actual M3 implementation and observed consequences.

2.4 ADR later-consequence updates

ADRs should not stop at documenting an initial decision. They should also record what happened when the decision was implemented.

ADR	Later consequence to document	Required evidence	Follow-up
ADR-M2-01	Whether logical layers remained separate during implementation	Repository review and dependency inspection	Correct violations and update architecture documentation
ADR-M2-01	Whether the single-application approach remained practical	Build and deployment experience	Retain or reconsider the decision based on evidence
ADR-M2-02	Whether all status changes passed through the transition service	Code review and endpoint tests	Remove bypasses or document an approved alternative
ADR-M2-02	Whether invalid transitions were consistently rejected	Automated tests and API responses	Correct business-rule inconsistencies
ADR-M2-02	Whether audit and persistence behaviour remained consistent	Database and integration tests	Correct transaction or audit-history problems
ADR-M2-02	Whether the chosen design remained understandable and maintainable	Developer review and implementation complexity	Refactor or revise the ADR if justified

ADR update template

For each later consequence, the following record should be completed:

- **Original decision:** What was agreed in M2?
- **Implementation evidence:** What was actually implemented?
- **Problem or consequence:** What worked, failed or became more difficult?
- **Revised decision:** Was the original decision retained or formally changed?
- **Reasoning:** Why was that decision appropriate?
- **Impact:** What are the effects on maintainability, testing, security, performance or delivery?
- **Follow-up:** What work remains, who is responsible and what evidence will close it?

This provides decision traceability and prevents undocumented changes from becoming accepted architecture by accident.

3. Implementation-Stage Risk Register

3.1 Purpose

The Risk Register extends the project's M2 risk-management work into implementation and release preparation.

Risks are evaluated according to their likelihood, impact and mitigation. The entries below are proposed implementation-stage risks to be reconciled with the controlled project register before final issue. They must not replace or duplicate existing risks without checking the original register.

A simple qualitative scale is used:

- **Likelihood:** Low, Medium or High.
- **Impact:** Low, Medium or High.
- **Priority:** High where a risk could compromise core business rules, security or release acceptance; Medium where the issue is significant but more contained.

3.2 Proposed implementation-stage risks

Risk ID	Risk description	Likelihood	Impact	Priority	Mitigation and response	Evidence for closure
R-046	Status changes may bypass the central transition service, producing inconsistent business behaviour	Medium	High	High	Review all status-writing paths and centralise transition validation	Code review and tests showing that every write path uses the approved logic
R-047	Invalid transitions may be accepted because the transition matrix is incomplete or incorrectly implemented	Medium	High	High	Define the approved transition matrix and test allowed and disallowed transitions	Passing transition tests for the documented rules
R-048	A status change may be persisted without the corresponding audit history, or history may be recorded incorrectly	Medium	High	High	Define audit requirements and test persistence and history together	Integration-test evidence and database inspection
R-049	Role-based restrictions may be applied inconsistently across endpoints	Medium	High	High	Apply authorisation at the appropriate server-side boundary and test permitted and forbidden actions	Role-based API tests and security review
R-050	Concurrent requests may overwrite changes or leave the record in an inconsistent state	Medium	High	High	Use suitable concurrency control or transactions and test competing updates	Concurrency-test results and database consistency checks
R-051	Logical architecture layers may become tightly coupled during implementation	Medium	Medium	Medium	Review dependencies, keep business logic out of controllers and document exceptions	Repository review and corrective-action records
R-052	Automated tests may pass without covering important integration or business-rule failures	Medium	High	High	Combine unit tests with integration, API and end-to-end checks	Test inventory and recorded results across the required test levels
R-053	The custom test harness may not be executed consistently by all contributors or CI	Medium	Medium	Medium	Document the command, prerequisites and expected output; integrate it into CI where supported	Reproducible local run and CI execution record

R-054	Deployment or environment configuration may prevent the application from starting in the target environment	Medium	High	High	Validate environment configuration and startup behaviour before release	Successful startup evidence from the intended target environment
R-055	Defects may remain unresolved because there is no agreed release-blocking severity policy	Medium	High	High	Define defect severity, ownership and release-blocking criteria	Reviewed defect register and explicit release decision
R-056	Acceptance requirements may be marked complete without evidence that they have been verified	Medium	High	High	Maintain requirement-to-test-to-result traceability	Acceptance matrix with evidence and reviewer confirmation
R-057	Performance requirements may not be tested under the intended load	Medium	Medium	Medium	Define the workload and collect measurements against the stated performance requirement	Reproducible performance-test report
R-058	Documentation may describe planned functionality rather than the final implementation	Medium	Medium	Medium	Review documentation against the M3 repository and observed behaviour	Documentation review and version alignment

Note: These are proposed entries, not verified additions to the controlled register. Confirm the next available IDs and merge them with the existing M2 risks before submission.

3.3 Risk treatment priorities

The highest priorities are risks that could cause invalid business state, unauthorised operations, inconsistent audit history or an unsupported release decision.

The recommended order of attention is:

1. Confirm that status-transition rules are centralised and correct.
2. Verify server-side authorisation and rejection behaviour.
3. Confirm that persistence and audit history remain consistent.
4. Execute automated and integration tests.
5. Confirm that the target environment can start and run the application.
6. Reconcile defects and acceptance requirements.
7. Complete performance and documentation checks appropriate to the release scope.

Risk treatment is complete only when the mitigation has been implemented and the agreed evidence has been reviewed. Writing a mitigation in the register does not prove that the risk has been resolved.

4. Architecture and Design Technical Debt Register

4.1 Purpose

Technical debt represents a design or implementation compromise that creates future maintenance cost, increases the likelihood of defects or makes verification more difficult.

The register distinguishes intentional trade-offs from accidental weaknesses. Technical debt should have an owner, a consequence and a proposed closure condition so that it can be managed rather than forgotten.

The entries below are proposed items based on the architectural concerns identified in this review. They must be checked against the existing project register to avoid duplicating recorded items.

4.2 Technical debt entries

ID	Technical debt	Consequence	Priority	Recommended action	Closure condition
TD-01	Logical layer boundaries may not be enforced by the project structure	Increases coupling and makes future changes harder to isolate	Medium	Review dependencies and introduce suitable checks or conventions	Dependencies reviewed and documented violations resolved
TD-02	Status-transition enforcement may be incomplete or bypassable	Different endpoints may allow different state changes	High	Route all status-changing operations through the approved transition logic	Code review and tests confirm consistent enforcement
TD-03	Test execution may depend on a custom harness without sufficiently standardised automation	Results may be difficult to reproduce or may be omitted from CI	Medium	Document test execution and integrate it into automation where feasible	Repeatable execution documented and automation evidence recorded
TD-04	Architectural decisions may lack implementation-stage consequence records	Future contributors may not understand why the design exists or why deviations occurred	Medium	Update ADRs with evidence, consequences and follow-up decisions	ADRs reviewed and aligned with the implementation
TD-05	Release readiness may rely on a checklist without linked evidence for each criterion	The release decision may be subjective or difficult to audit	High	Link each release criterion to a test, report, review or approval	Every required criterion has evidence or an explicit unresolved status

4.3 Managing technical debt

The recommended management process is:

1. Record the debt and the affected component.
2. Explain the reason it matters.
3. Estimate the likelihood and consequence of leaving it unresolved.
4. Identify a practical remediation.
5. Assign an owner and target milestone.
6. Define objective closure evidence.
7. Reassess the priority when the implementation or release scope changes.

Not all technical debt must be eliminated immediately. However, debt affecting security, core business rules, data consistency or release-critical functionality should receive higher priority than purely cosmetic or low-impact refactoring.

5. Status-Transition Test Specifications

5.1 Purpose and testing approach

The purpose of these test specifications is to verify that CivicConnect handles status changes according to its approved business rules.

The tests are intended to cover the central transition logic and the boundaries around it. They include valid transitions, invalid transitions, authorisation, persistence, audit history and concurrent updates.

These are **test specifications, not recorded test results**. The approved transition matrix, status names, role names, API routes and expected response codes must be aligned with the actual CivicConnect requirements and implementation before execution.

5.2 Test assumptions

For the specifications below:

- The system has a defined set of statuses.
- The application has an approved transition matrix.
- The server is responsible for validating status changes.
- The caller's permissions can affect whether an operation is permitted.
- The application persists accepted changes.
- Audit history is required where specified by the project requirements.
- A rejected operation should not leave a partially applied change.

Where a specific status or role is not stated in the available baseline, the test uses a placeholder that must be replaced with the approved project value.

5.3 Test case specifications

TC-ST-01: Valid status transition

Objective: Verify that an allowed transition is accepted.

Preconditions:

- A record exists in a valid initial status.
- The approved transition matrix permits the requested target status.
- The caller is authorised to perform the operation.

Steps:

1. Retrieve the record's current status.
2. Submit a request to change it to an allowed target status.
3. Inspect the response.
4. Retrieve the record again.

Expected result:

- The operation is accepted.
- The response communicates success.
- The persisted status matches the requested target status.
- Any required audit history is recorded.

TC-ST-02: Invalid status transition

Objective: Verify that a transition not permitted by the approved matrix is rejected.

Preconditions:

- A record exists in a status from which the requested target is not permitted.

Steps:

1. Record the current status.
2. Submit the prohibited transition.
3. Inspect the response.
4. Retrieve the record again.

Expected result:

- The operation is rejected.
- The response communicates that the transition is invalid.
- The persisted status remains unchanged.
- No misleading success audit event is recorded.

TC-ST-03: Transition from each supported status

Objective: Verify that the configured transition rules are applied consistently across supported statuses.

Preconditions:

- The approved transition matrix is available.

Steps:

1. Prepare a record for each supported current status.
2. Attempt each relevant target status.
3. Compare each outcome with the approved transition matrix.

Expected result:

- Every allowed transition is accepted when all other conditions are satisfied.
- Every prohibited transition is rejected.
- The observed behaviour matches the approved matrix.

TC-ST-04: Missing target status

Objective: Verify that a request without a target status is rejected.

Steps:

1. Submit a status-change request with the target status omitted or null, where the API permits the request to reach validation.
2. Inspect the response.
3. Retrieve the record.

Expected result:

- The request is rejected with an appropriate validation response.
- The existing status remains unchanged.
- The application does not create an incomplete transition.

TC-ST-05: Unknown or unsupported target status

Objective: Verify that the application rejects a status value not recognised by the domain model.

Steps:

1. Submit a transition request containing an unsupported status value.
2. Inspect the response.
3. Retrieve the record.

Expected result:

- The request is rejected.
- The unsupported value is not persisted as a valid status.
- The record remains in its original state.

TC-ST-06: Unauthorised user attempts a transition

Objective: Verify that an unauthorised caller cannot change a record's status.

Preconditions:

- A record exists.
- The caller does not have the permission required for the requested operation.

Steps:

1. Authenticate as the restricted user.
2. Submit an otherwise valid transition request.
3. Inspect the response.
4. Retrieve the record using an authorised test identity.

Expected result:

- The operation is denied.
- The record's status remains unchanged.
- The denial is handled according to the application's security and logging requirements.

TC-ST-07: Authorised user performs a transition

Objective: Verify that a user with the required permission can perform an otherwise valid transition.

Preconditions:

- The record is in an eligible initial state.
- The user has the required permission.

Steps:

1. Authenticate as the authorised user.
2. Submit an allowed transition.
3. Inspect the response.
4. Retrieve the record.

Expected result:

- The request is accepted if all business conditions are satisfied.
- The new status is persisted.
- The operation follows the same transition rules as other authorised entry points.

TC-ST-08: Rejected transition does not alter the record

Objective: Verify that a failed transition does not partially modify the record.

Steps:

1. Record the current status and relevant record fields.
2. Submit a transition that should be rejected.
3. Retrieve the record and compare it with the original state.

Expected result:

- The status remains unchanged.
- No unrelated fields are unintentionally modified.
- No partial update is left behind.

TC-ST-09: Successful transition creates required audit history

Objective: Verify that a successful transition records the required history.

Preconditions:

- Audit history is part of the approved requirements.
- The record is eligible for the transition.

Steps:

1. Record the existing audit history.
2. Submit an allowed transition.
3. Retrieve the audit history.
4. Compare the new history with the operation performed.

Expected result:

- The new status is persisted.
- The required audit event is recorded.
- The history accurately identifies the transition and any required actor and timestamp information.
- The audit entry corresponds to the actual successful operation.

TC-ST-10: Rejected transition does not create misleading success history

Objective: Verify that an invalid or unauthorised transition does not appear as a successful change.

Steps:

1. Record the current status and audit history.
2. Submit a request that must be rejected.
3. Inspect the response and retrieve the audit history.

Expected result:

- The status remains unchanged.
- No success event is recorded for a transition that did not occur.
- Any security or rejection logging follows the approved logging requirements.

TC-ST-11: Transition validation occurs before persistence

Objective: Verify that business validation prevents invalid changes from reaching persistent storage.

Steps:

1. Prepare a record that cannot legally enter the requested target status.
2. Submit the invalid transition.
3. Inspect the response and retrieve the record directly through the supported data-access or retrieval mechanism.

Expected result:

- Validation rejects the request.
- The invalid target status is not persisted.
- The stored record remains consistent.

TC-ST-12: Transition rules are enforced across entry points

Objective: Verify that different supported endpoints or application entry points cannot bypass the central transition rules.

Preconditions:

- The application has more than one relevant route or entry point capable of requesting status changes.

Steps:

1. Identify all supported status-changing entry points.
2. Submit the same prohibited transition through each applicable entry point.
3. Compare responses and persisted state.

Expected result:

- All entry points enforce the approved business rules.
- No route permits a transition that the central rules prohibit.
- The persisted state remains consistent.

TC-ST-13: Persistence failure during a transition

Objective: Verify that a persistence failure does not leave the application reporting a successful transition that was not stored correctly.

Preconditions:

- A controlled test environment can simulate or safely reproduce a persistence failure.

Steps:

1. Prepare a record eligible for a valid transition.
2. Trigger a controlled persistence failure during the operation.
3. Inspect the response.
4. Retrieve the record and any associated audit history.

Expected result:

- The application does not report an unqualified success if the required update failed.
- The stored record remains consistent with the transaction outcome.
- Audit history and status data do not contradict one another.
- The error is handled according to the application's failure-handling requirements.

TC-ST-14: Concurrent status-transition requests

Objective: Verify that concurrent requests do not silently create an invalid final state.

Preconditions:

- A record exists in a known initial status.
- Two competing requests can be issued in a controlled test environment.

Steps:

1. Prepare two requests based on the same initial record state.
2. Submit the requests concurrently or with controlled overlap.
3. Inspect both responses.
4. Retrieve the final record and audit history.

Expected result:

- The system follows its defined concurrency strategy.
- The final state is valid under the approved transition rules.
- A conflicting request is rejected or handled through the documented concurrency mechanism.
- The final record and audit history remain consistent.

5.4 Test coverage summary

Coverage area	Test cases
Valid transitions	TC-ST-01, TC-ST-03, TC-ST-07
Invalid or unsupported input	TC-ST-02, TC-ST-04, TC-ST-05
Authorisation	TC-ST-06, TC-ST-07
Rejected-operation integrity	TC-ST-08, TC-ST-10, TC-ST-11
Audit history	TC-ST-09, TC-ST-10
Central enforcement	TC-ST-12
Persistence failure	TC-ST-13
Concurrency	TC-ST-14

The test specifications cover the main failure paths associated with status changes. Before execution, the team must map each test to the exact requirements and implementation interfaces. Tests must then be executed and their actual outcomes recorded.

5.5 Test evidence and defect handling

For every executed test, the test record should include:

- Test case ID.
- Requirement or rule being verified.
- Build or source-code version.
- Test environment.
- Preconditions and test data.
- Actual steps performed.
- Expected result.
- Actual result.
- Pass, fail or blocked status.
- Defect reference, where applicable.
- Evidence such as logs, screenshots or test output.
- Tester and execution date.

A test must not be marked as passed merely because its expected result has been written down. A test is passed only when it has been executed and the observed result matches the expected result.

6. Release Evidence Summary

6.1 Purpose

The Release Evidence Summary consolidates the evidence needed to decide whether CivicConnect is ready for release.

The release decision must be based on implementation and verification evidence rather than the apparent completeness of documentation. A feature may be documented and designed but still require implementation, integration or acceptance testing.

6.2 Release-readiness classifications

READY

The release may be classified as READY when all mandatory acceptance criteria have been verified, required tests have passed, critical and release-blocking defects have been resolved, the target environment has been validated, and the required approvals are recorded.

CONDITIONALLY READY

The release may be classified as CONDITIONALLY READY when the essential release requirements are satisfied but a limited number of non-critical issues remain. Each remaining issue must have an owner, a documented impact, an agreed action and an authorised acceptance of the risk.

This classification must not be used to overlook unresolved critical security, data-integrity or core-business-rule failures.

NOT READY

The release is NOT READY when required functionality is incomplete, mandatory evidence is missing, important tests have failed or not been executed, the target environment is unverified, or unresolved defects create unacceptable risk.

6.3 Release evidence checklist

Evidence area	Required evidence	Current assessment based on reviewed material
Build	Reproducible build output from the release candidate	Pending verification
Automated tests	Recorded execution of the automated test suite	The reviewed baseline describes a custom C# harness with 29 checks; a successful M3 run must be evidenced
CI	A recorded CI run for the relevant revision	A `verify.yml` file was present in the reviewed baseline, but its presence alone does not prove a successful remote run
Unit tests	Results for status-transition business rules	Test specifications prepared; execution results pending
Integration tests	Evidence that API, business logic and persistence work together	Pending verification
End-to-end tests	Evidence of key user workflows in the integrated application	Pending verification
Security and authorisation	Evidence that restricted operations are denied and permitted operations work correctly	Pending verification
Data integrity	Evidence that rejected transitions and failures do not leave inconsistent data	Pending verification

Performance	Measured results against the applicable non-functional requirement	Pending verification
Defects	Current defect list, severity, ownership and release impact	Pending verification
Acceptance criteria	Requirement-to-test traceability and acceptance evidence	Pending verification
Environment and configuration	Evidence that the application starts and operates in the intended target environment	The reviewed baseline indicates startup is refused outside Development; the intended deployment configuration must be verified
Documentation	Documentation matches the final implementation and release	Pending final review
Approval	Recorded release decision and any accepted residual risks	Pending verification

6.4 Performance evidence

The reviewed project material refers to performance requirement NFR-002, with a workload described as 10 users, 1,000 requests and a 10-minute period.

The exact performance acceptance threshold and measurement method must be taken from the approved requirement. The team should not invent a response-time target or claim that the requirement has been met without recorded measurements.

The performance report should record:

- The build and environment tested.
- The workload and test duration.
- The number of requests completed and failed.
- Response-time measurements.
- Resource usage where relevant.
- Errors or timeouts.
- Whether the approved acceptance threshold was met.

6.5 Release decision

Based on the reviewed baseline information alone, **release readiness has not been demonstrated**. The existence of a test harness, a CI configuration file or documentation is not sufficient to establish that the release candidate passes the required checks.

The appropriate preliminary classification is:

NOT READY — pending verification of mandatory M3 implementation and release evidence.

This is a preliminary evidence-based assessment, not a claim that the final M3 implementation has failed. The classification can change when the required implementation, test, environment and acceptance evidence has been reviewed.

6.6 Conditions for changing the classification

Before a release can be reconsidered, the team should:

1. Confirm that the required M3 functionality is implemented.
2. Run the automated test suite and retain its output.
3. Execute the status-transition tests.
4. Complete the relevant integration and end-to-end checks.
5. Verify authorisation and data-integrity behaviour.
6. Confirm the target environment starts and operates correctly.
7. Review performance evidence against the approved requirement.
8. Resolve or formally assess release-blocking defects.
9. Link acceptance requirements to actual verification evidence.
10. Record the release decision and any accepted residual risks.

The final classification must reflect the evidence available at the time of the release review.

7. Residual Risk Statement

Even after corrective actions and verification activities have been completed, CivicConnect may retain some residual risk.

Potential residual risks include:

- Unexpected combinations of status transitions that are not represented in the test data.
- Concurrency problems that occur only under specific timing conditions.
- Differences between development and target deployment configuration.
- Changes introduced after the last successful test run.
- Architectural dependencies that are not automatically enforced.
- Performance behaviour that differs under workloads outside the tested conditions.
- Defects that remain undiscovered despite the planned test coverage.

These risks can be reduced through centralised business-rule enforcement, repeatable automated tests, integration testing, appropriate concurrency controls, environment verification, code review and traceable release decisions.

Residual risk must be evaluated against the actual release evidence. If a risk could affect security, data integrity, core status-transition rules or mandatory acceptance criteria, it should not be accepted solely because the deadline is approaching.

Any residual risk accepted for release should have a documented description, impact, mitigation, owner and explicit acceptance by the appropriate project authority.

Residual risk conclusion: CivicConnect should be released only when the mandatory requirements have been verified and the remaining risks are understood and formally accepted. Where required evidence is missing, the uncertainty itself must be treated as a release concern.

8. Individual Defence Preparation

This section explains the reasoning behind my contribution and prepares me to discuss the decisions during an individual defence.

8.1 Dimension 1: Requirements and evidence traceability

Question: How did you ensure that the M3 work was traceable to the project requirements?

My approach was to link architectural decisions to implementation observations, implementation risks to corrective actions, and test specifications to the business rules they are intended to verify. The release checklist then uses verification evidence to assess whether the required functionality is ready.

Traceability is important because a requirement should not be considered complete merely because it appears in documentation. There should be evidence that the corresponding behaviour exists and works as expected.

Question: What is the difference between a requirement, a test specification and a test result?

A requirement describes what the system must do. A test specification describes how that behaviour will be checked and what outcome is expected. A test result records what actually happened when the test was executed.

These are related but distinct artefacts. A completed test specification does not establish that the implementation passed the test.

8.2 Dimension 2: Engineering decisions

Question: Why retain a single deployable application?

A single application reduces deployment and operational complexity for a project of this scale. Logical layers can still provide separation of responsibilities without introducing the additional communication, monitoring and deployment overhead of multiple services.

This decision should be reconsidered only if the requirements or implementation experience demonstrate a concrete need for independent deployment, scaling or stronger service-level separation.

Question: Why is centralised status-transition logic important?

Status transitions represent business rules, not merely changes to a database field. Centralising the logic reduces duplication and ensures that different endpoints apply the same rules.

It also provides one place to validate transitions, apply relevant business constraints and coordinate persistence and audit behaviour.

Question: What would justify revising an ADR?

An ADR should be revised when implementation evidence or changed requirements demonstrate that the original decision is no longer appropriate, or when a deliberate alternative provides a better trade-off.

A revision should explain the original decision, the evidence that prompted reconsideration, the alternatives evaluated, the new decision and its consequences. It should not silently replace the earlier reasoning.

8.3 Dimension 3: Quality and verification

Question: Why are both unit and integration tests necessary?

Unit tests can verify the transition rules in isolation, including allowed and prohibited state changes. Integration tests are needed to establish that the business logic works correctly with the API, persistence mechanism, authorisation and audit history.

A unit test can pass while the integrated application still fails because of configuration, database or endpoint behaviour.

Question: Why test invalid transitions if valid transitions already work?

Invalid-transition tests verify the boundaries of the business rules. A system that accepts valid transitions but also accepts prohibited transitions is not implementing the rules correctly.

Negative tests also help demonstrate that rejected operations do not change the record or create misleading audit history.

Question: Why include concurrent requests?

Two requests can act on the same record before either operation has completed. Without suitable concurrency handling, one request may overwrite another or produce an invalid final state.

The concurrency test checks whether the application follows its documented strategy when competing requests occur.

8.4 Dimension 4: Failure and risk reasoning

Question: Which risks should receive the highest priority?

Risks affecting core business rules, authorisation, data integrity and release-critical functionality should receive the highest priority because their consequences can extend beyond a single screen or request.

For example, a status-transition bypass could cause inconsistent behaviour across the application. An authorisation failure could permit a user to perform an operation they should not be able to perform. An audit inconsistency could undermine the reliability of the record's history.

Question: How do you know whether a mitigation has worked?

The risk should be linked to objective closure evidence. If the mitigation is to centralise transition logic, closure requires more than documenting the change: code review and tests should demonstrate that the relevant entry points use the central rules.

If the mitigation is to improve CI, the presence of a configuration file is not enough. A recorded execution and result are needed.

Question: Why document technical debt rather than simply fixing every issue immediately?

Some technical debt may be acceptable temporarily when the trade-off is understood and the risk is manageable. The register makes that debt visible and provides a basis for prioritisation.

However, debt affecting security, business-rule correctness, data consistency or release-critical behaviour should not be deferred without an explicit and defensible decision.

8.5 Dimension 5: Release readiness

Question: Why is the preliminary release classification NOT READY?

The reviewed evidence does not establish that the mandatory M3 implementation, automated tests, integration behaviour, target environment and acceptance requirements have all been verified.

This does not prove that the final implementation is defective. It means the available evidence is insufficient to justify a positive release-readiness decision.

Question: What would change the release classification?

The classification could change after the team supplies evidence of the completed implementation, successful required tests, verified target-environment behaviour, acceptable defect status and satisfied acceptance criteria.

The decision should be made from the complete release evidence rather than from one successful test or the apparent completeness of the documentation.

Question: What is the difference between CONDITIONALLY READY and NOT READY?

CONDITIONALLY READY means the essential requirements are satisfied and only limited, non-critical issues remain that can be managed through explicit conditions and accepted risk.

NOT READY means a mandatory requirement remains unverified or unsatisfied, a critical failure remains unresolved, or the available evidence is insufficient to support release.

8.6 Personal contribution summary

My contribution focuses on making architectural decisions, implementation risks and quality evidence traceable. I have structured the architecture-conformance review around the M2 decisions, identified the evidence required to assess the M3 implementation, proposed implementation-stage risks and technical-debt entries, and prepared status-transition test specifications.

I have also established a release-readiness approach that distinguishes between completed documentation and verified implementation. This is important because the project needs a defensible release decision based on evidence rather than assumptions.

My work supports the development team by clarifying what must be reviewed, what should be tested, which failures matter most and what evidence is required before the release can be considered ready.

9. Final Conclusion

My M3 contribution provides a structured approach to architecture conformance, decision traceability, implementation-stage risk management, technical debt, status-transition verification and release readiness.

The architecture review identifies the evidence needed to determine whether the M2 decisions remain consistent with the implementation. The risk and technical-debt registers make potential weaknesses visible and link them to corrective actions. The status-transition test specifications cover valid behaviour, rejected operations, authorisation, persistence, audit history and concurrency. The release evidence checklist defines the information needed to support a defensible release decision.

The main conclusion is that release readiness must be demonstrated through implementation and verification evidence. Architectural intentions, planned tests and existing configuration files are useful starting points, but they cannot substitute for observed results.

The final M3 review should therefore confirm the actual implementation, reconcile the proposed register entries with the controlled project records, execute the required tests, record outstanding defects and update the ADRs with the consequences observed during implementation.